

Anforderungsspezifikation StudentGo

Mobile Campus-App fuer Studierende der HFU

Produkt	StudentGo
Version	1.0
Status	Entwurf
Datum	1. Mai 2026
Bezug	Expo/React Native App mit Express/Prisma Backend

Inhaltsverzeichnis

1	Zweck und Geltungsbereich	2
1.1	Ziele	2
1.2	Systemkontext	2
2	Stakeholder und Rollen	2
3	Funktions-Master	2
4	User-Stories	3
5	Nichtfunktionale Anforderungen	6
6	4+1 Sichtenmodell	6
6.1	Use-Case-Sicht	6
6.2	Logische Sicht	7
6.3	Prozesssicht	7
6.4	Entwicklungssicht	7
6.5	Physische Sicht	8
7	Datenanforderungen	8
8	Schnittstellen	8
9	Abnahmekriterien	9
10	PlantUML-Integration	9
A	PlantUML-Quellen	10
A.1	Use-Case-Diagramm	10
A.2	Logische Komponenten	11
A.3	Offline-Prozess	11

1 Zweck und Geltungsbereich

Diese Anforderungsspezifikation beschreibt die fachlichen und technischen Anforderungen an StudentGo. Die App bundelt studienrelevante Alltagsfunktionen: Stundenplan, Mensa, Kontakte, To-dos, private Notizen, Rollenverwaltung und kontrollierte Inhaltsaenderungen. Sie basiert auf der bestehenden Codebasis mit Expo Router, React Native, Express, Prisma und SQLite.

1.1 Ziele

- Studierende erhalten einen kompakten, mobilen Zugriff auf persoenliche und oeffentliche Campus-Informationen.
- Managerinnen und Manager koennen Aenderungen fuer verwaltete Inhalte vorbereiten, ohne direkt produktive Daten zu ueberschreiben.
- Administratorinnen und Administratoren pruefen Antraege, verwalten Benutzer und importieren externe Datenquellen.
- Kerninformationen bleiben im Offline-Fall lesbar; neue persoenliche Eintraege werden nach Verbindungswiederkehr synchronisiert.

1.2 Systemkontext

StudentGo besteht aus einer mobilen bzw. webbasierten Expo-App, einem Express-Backend und einer SQLite-Datenbank. Externe Systeme sind der HFU-StarPlan fuer Stundenplandaten und der SWFR-Speiseplan fuer Mensadaten.

2 Stakeholder und Rollen

Rolle	Interesse
Student:in	Schneller Zugriff auf Stundenplan, Aufgaben, Notizen, Kontakte, Mensa und geteilte Veranstaltungen.
Manager:in	Erstellung von Aenderungsantraegen fuer Kontakte, Mensa, Lehrveranstaltungen und Studieninformationen.
Administrator:in	Benutzerverwaltung, Rollensteuerung, Freigabe oder Ablehnung von Aenderungen, Importsteuerung.
Backend-Betreiber:in	Sicherer und wartbarer Betrieb der API, Datenbank und Importprozesse.

3 Funktions-Master

ID	Funktion	Beschreibung	Prioritaet
FM-01	Registrierung und Login	Benutzer registrieren sich mit Name, E-Mail, Passwort und Rolle; Accounts werden ueber Bestaetigungstoken aktiviert.	Muss
FM-02	Rollenbasierter Zugriff	Sessions enthalten eine aktive Rolle; Admin darf alle Rollen nutzen, Manager keine Admin-Rolle, User nur User.	Muss
FM-03	Benutzerverwaltung	Admins legen Benutzer an, aendern Rollen, aktualisieren Accounts und loeschen Benutzer.	Muss

ID	Funktion	Beschreibung	Prioritaet
FM-04	Aenderungsworflow	Manager und Admins erstellen Aenderungsantraege; Admins genehmigen oder lehnen diese ab.	Muss
FM-05	Kontakte	Oeffentliche und persoenliche Kontakte anzeigen, suchen und persoenliche Kontakte anlegen.	Muss
FM-06	To-dos	Persoenliche To-dos mit Unteraufgaben anlegen, anzeigen und abschliessen.	Muss
FM-07	Private Notizen	Studieninformationen und persoenliche Notizen anzeigen; private Inhalte koennen clientseitig verschluesselt werden.	Soll
FM-08	Stundenplan	Wochenplan anzeigen, manuelle Veranstaltungen pflegen, Besuche markieren und Modul-Praeferenzen speichern.	Muss
FM-09	StarPlan-Import	Importoptionen laden, Kurse auswaehlen und Veranstaltungen aus HFU-StarPlan uebernehmen.	Soll
FM-10	Einladungen	Veranstaltungen mit anderen Benutzern teilen; Empfaenger koennen annehmen oder ablehnen.	Soll
FM-11	Mensa	Mensen und Speiseplaene anzeigen; Admin kann SWFR-Import ausloesen.	Soll
FM-12	Offline-Faehigkeit	GET-Antworten cachen, ausgewaehlte POST-Aktionen lokal vormerken und spaeter wiederholen.	Muss
FM-13	Theme und Bedienung	Helles/dunkles Erscheinungsbild, Statusanzeigen und Swipe-Navigation bereitstellen.	Kann

4 User-Stories

ID	Story	Akzeptanzkriterien
US-01	Als Student:in moechte ich mich registrieren und bestaetigen, damit ich persoenliche Daten speichern kann.	Bestaetigungstoken wird erzeugt; vor Bestaetigung existiert nur ein PendingAccount; nach Bestaetigung ist Login moeglich.

ID	Story	Akzeptanzkriterien
US-02	Als Student:in moechte ich meine Kontakte durchsuchen, damit ich Personen schnell finde.	Suche filtert Name, Rolle, E-Mail und Raum; persoenliche Kontakte werden markiert.
US-03	Als Student:in moechte ich To-dos mit Unteraufgaben erfassen, damit ich Studienaufgaben strukturieren kann.	To-do wird dem Account zugeordnet; Unteraufgaben sind separat abschliessbar; erledigte Aufgaben werden erkennbar getrennt.
US-04	Als Student:in moechte ich private Notizen verschluesseln, damit sensible Inhalte nur mit meinem Private Key lesbar sind.	Public Key wird am Account gespeichert; verschluesselte Payloads enthalten Key und IV; ohne passenden Private Key wird ein Hinweis angezeigt.
US-05	Als Student:in moechte ich meinen Stundenplan importieren, damit ich ihn nicht manuell uebertragen muss.	Importoptionen werden geladen; Auswahl erzeugt Lessons und ScheduleDays; Duplikate werden anhand stabiler Modulschlüssel vermieden.

ID	Story	Akzeptanzkriterien
US-06	Als Student:in moechte ich eine Veranstaltung teilen, damit Kommiliton:innen sie uebernehmen koennen.	Einladung referenziert Lesson, Sender und Empfaenger; Status ist initial PENDING; Annahme/Ab- lehnung wird gespeichert.
US-07	Als Manager:in moechte ich Aenderungen beantragen, damit fachliche Inhalte kontrolliert gepflegt werden.	Antrag enthaelt Entity, Action, Payload und optional TargetId; Status ist PENDING; eigene Antraege sind sichtbar.
US-08	Als Admin moechte ich Aenderungsantraege freigeben, damit produktive Daten erst nach Pruefung geaendert werden.	Approve wendet Payload auf Zielentitaet an; Reject speichert Review-Notiz; Status und Review-Zeitpunkt werden gesetzt.
US-09	Als Admin moechte ich Benutzerrollen verwalten, damit Verantwortlichkeiten korrekt abgebildet sind.	Rollen USER, MANAGER und ADMIN sind wahlbar; unzuessaessige Rollenwechsel werden serverseitig verhindert.
US-10	Als Nutzer:in moechte ich bei Netzproblemen weiterarbeiten, damit kurze Verbindungsabbrueche mich nicht blockieren.	Cache liefert bekannte GET-Daten; lokale Creates erhalten Pending-Status; Sync-Context spielt Requests spaeter ab.

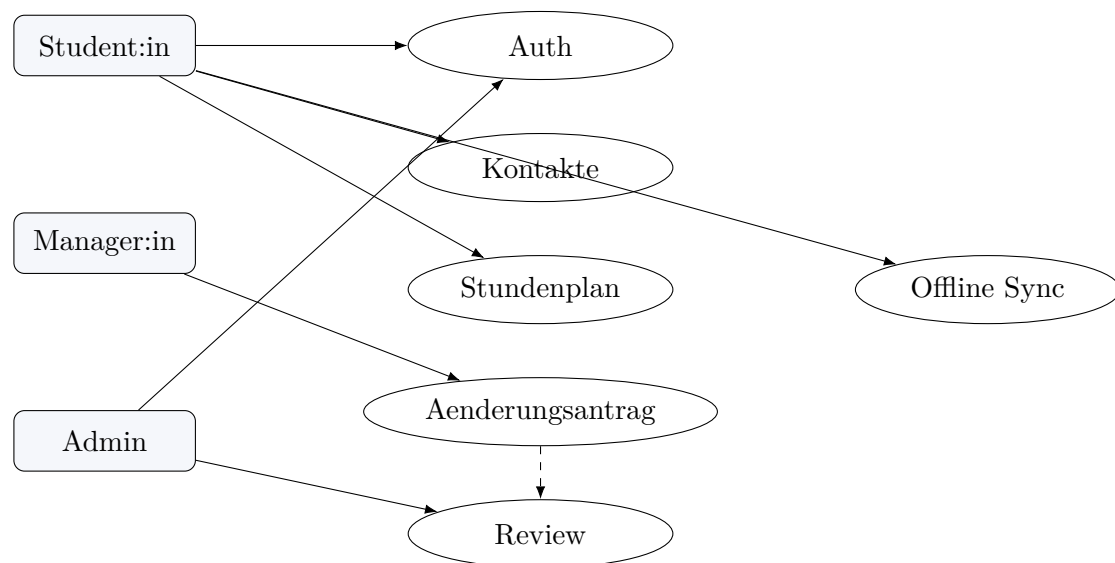
5 Nichtfunktionale Anforderungen

ID	Kategorie	Anforderung
NF-01	Sicherheit	Passwoerter werden mit Salt und scrypt gehasht; Sessiontokens werden nur als SHA-256 Hash gespeichert.
NF-02	Datenschutz	Private Schluessel verbleiben clientseitig; verschluesselte Inhalte werden als Payload, Key und IV gespeichert.
NF-03	Zugriffsschutz	Admin- und Manager-Endpunkte muessen serverseitig ueber Middleware geschuetzt sein.
NF-04	Offline	Cache und Queue duerfen Benutzerkontext nicht vermischen; Cache-Key beruecksichtigt URL und Authorization-Header.
NF-05	Wartbarkeit	Fachliche Modelle werden zentral in Prisma beschrieben; API und App verwenden klar benannte Module und Contexts.
NF-06	Portabilitaet	App laeuft auf Expo-Zielplattformen Android, iOS und Web; Backend laeuft lokal oder auf Node.js Hosts.
NF-07	Nachvollziehbarkeit	Aenderungsantraege speichern Ersteller, Review-Daten, Status und Payload.
NF-08	Importrobustheit	Externe Importe muessen fehlschlagen duerfen, ohne bestehende App-Funktionen zu blockieren.

6 4+1 Sichtenmodell

6.1 Use-Case-Sicht

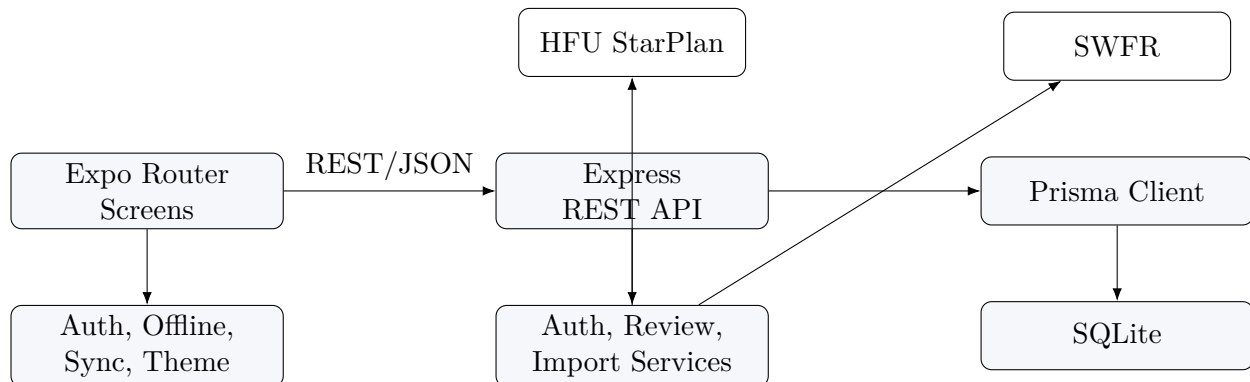
Die Use-Case-Sicht beschreibt die wichtigsten Interaktionen zwischen Rollen und System.



PlantUML-Quelle: `diagrams/usecase.puml`.

6.2 Logische Sicht

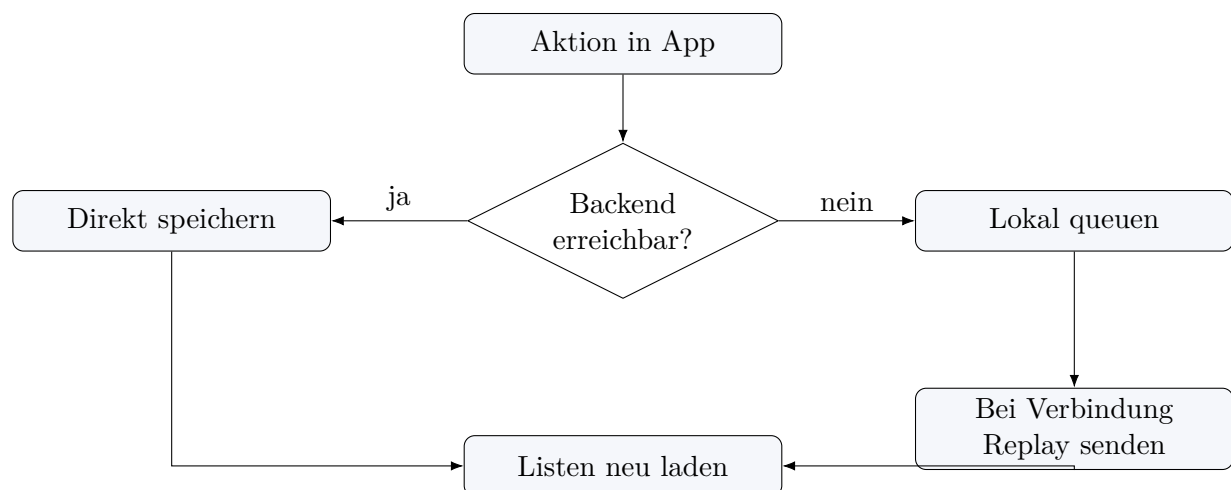
Die logische Sicht trennt App-Kontexte, Backend-Services und Persistenz.



PlantUML-Quelle: `diagrams/logical-components.puml`.

6.3 Prozesssicht

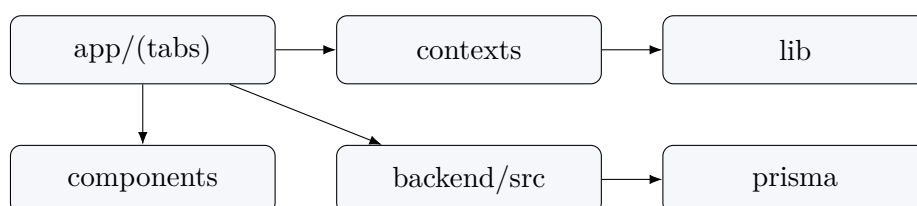
Wichtige Laufzeitprozesse sind Login, Offline-Synchronisation, Aenderungsreview, StarPlan-Import und SWFR-Import. Die Offline-Synchronisation puffert schreibende Aktionen fuer Kontakte und To-dos, markiert sie lokal als pending und spielt sie mit Replay-Header erneut ab.



PlantUML-Quellen: `diagrams/process-sync.puml`, `diagrams/admin-review-sequence.puml`, `diagrams/scenario-schedule-import.puml`.

6.4 Entwicklungssicht

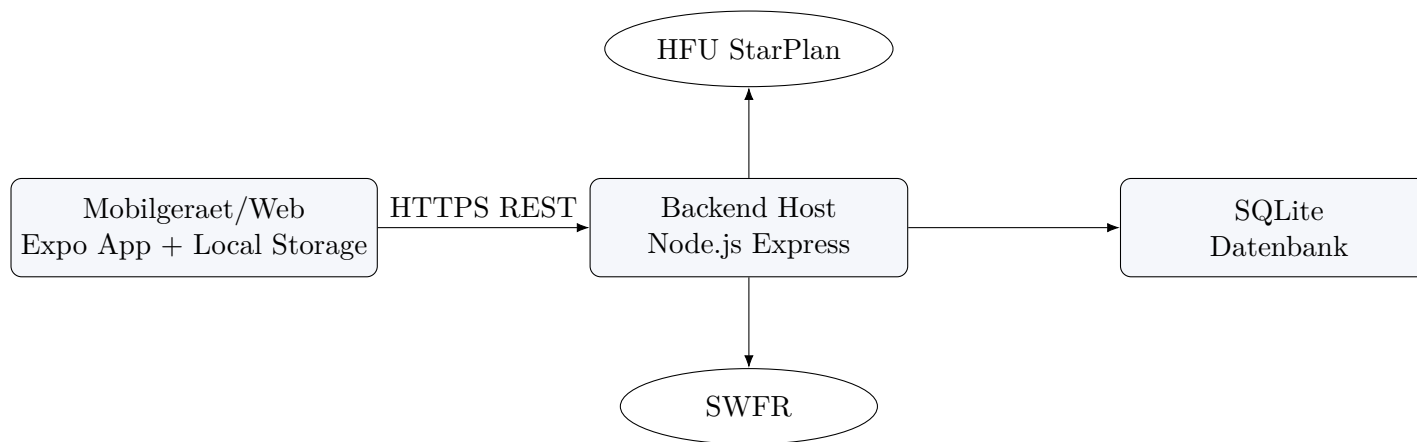
Die Entwicklungssicht folgt der vorhandenen Projektstruktur. Screens liegen in `app/(tabs)`, gemeinsam genutzte Zustandslogik in `contexts`, plattformspezifische Persistenz und Kryptografie in `lib`, Backend-Code in `backend/src` und Datenmodell/Migrationen in `prisma`.



PlantUML-Quelle: `diagrams/development-view.puml`.

6.5 Physische Sicht

Die physische Sicht besteht aus Clientgeraet, Backend-Host, Datenbank und externen Importquellen.



PlantUML-Quelle: `diagrams/deployment-view.puml`.

7 Datenanforderungen

Das zentrale Datenmodell wird durch Prisma beschrieben. Besonders relevant sind **User**, **Session**, **PendingAccount**, **ManagementChangeRequest**, **Contact**, **Todo**, **StudyInfo**, **Lesson**, **LessonInvitation**, **Canteen** und **MealPlan**.

Entitaet	Zweck
User	Account, Rolle, Public Keys und Beziehungen zu persoenlichen Daten.
Session	Tokenbasierte Anmeldung mit aktiver Rolle und Ablaufzeit.
ManagementChangeRequest	Auditierbarer Aenderungsantrag mit Action, Entity, Payload und Review-Daten.
Lesson	Einzelne oder wiederkehrende Veranstaltung inklusive Importquelle, Raum, Dozent:in und Verschluesselungsfeldern.
ScheduleImportCache	Zwischenspeicher fuer importierte StarPlan-Daten pro Woche und Auswahl.
StudyInfo	Oeffentliche Studieninformationen oder persoenliche, optional verschluesselte Notizen.

PlantUML-Quelle: `diagrams/data-model.puml`.

8 Schnittstellen

Endpoint	Zweck	Schutz
POST /api/auth/register	Registrierung und PendingAccount-Erzeugung.	oeffentlich
POST/GET /api/auth/confirm	Accountbestaetigung per Token.	oeffentlich
POST /api/auth/login	Session erzeugen.	oeffentlich

Endpoint	Zweck	Schutz
GET /api/auth/me	Aktuellen Benutzer laden.	Token
GET/POST /api/contacts	Kontakte lesen und persoenliche Kontakte anlegen.	optional/Token
GET/POST /api/todos	Persoenliche To-dos lesen und anlegen.	Token
GET/POST /api/study-info	Studieninfos und Notizen lesen/anlegen.	optional/Token
GET /api/schedule	Stundenplan fuer Woche/Monat laden.	optional/Token
POST /api/schedule/import	StarPlan-Veranstaltungen importieren.	Token
POST /api/schedule/lessons/:id/visit	Besuch einer Veranstaltung markieren.	Token
GET/POST /api/manager/change-requests	Aenderungsantraege lesen/anlegen.	Manager/Admin
GET /api/admin/users	Benutzerliste lesen.	Admin
POST /api/admin/change-requests/:id/approve	Antrag genehmigen.	Admin
POST /api/admin/import/swfr-meals	Mensa-Import ausloesen.	Admin

9 Abnahmekriterien

- Die App kann mit lokalem Backend starten und /health erfolgreich abfragen.
- Registrierung, Bestaetigung, Login, Logout und Rollenwahl funktionieren gemaess Zugriffsmatrix.
- Admin-Endpunkte sind ohne Admin-Session nicht erreichbar.
- Kontakte und To-dos koennen offline vorgemerkt und nach Wiederverbindung synchronisiert werden.
- StarPlan- und SWFR-Importfehler fuehren zu sichtbarer Fehlermeldung, aber nicht zum Absturz der App.
- Aenderungsantraege koennen erstellt, genehmigt, abgelehnt und im Status nachvollzogen werden.
- PlantUML-Quellen liegen unter diagrams/ und koennen mit PlantUML in Diagrammartefakte uebersetzt werden.

10 PlantUML-Integration

Die UML-Quellen liegen als echte PlantUML-Dateien im Ordner diagrams/. Damit LaTeX PlantUML direkt parsen kann, ist zusaetzlich plantuml-latex-demo.tex enthalten. Dieser Build nutzt das lokal installierte LaTeX-Paket plantuml; erforderlich sind lualatex -shell-escape und die Umgebungsvariable PLANTUML_JAR.

```
export PLANTUML_JAR=/pfad/zu/plantuml.jar
make plantuml-latex
```

Der Standard-Build `make pdf` erzeugt diese Spezifikation ohne externes Jar. Sobald PlantUML installiert ist, erzeugt `make plantuml` zusätzlich gerenderte Diagramme aus den `.puml`-Dateien.

A PlantUML-Quellen

A.1 Use-Case-Diagramm

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Student:in" as Student
actor "Manager:in" as Manager
actor "Administrator:in" as Admin
actor "SWFR Mensa" as SWFR
actor "HFU StarPlan" as StarPlan

rectangle "StudentGo" {
    usecase "Registrieren und anmelden" as UCAuth
    usecase "Rolle aktivieren" as UCRole
    usecase "Kontakte suchen und pflegen" as UCContacts
    usecase "To-dos und private Notizen verwalten" as UCTodos
    usecase "Stundenplan anzeigen" as UCSchedule
    usecase "Stundenplan importieren" as UCImport
    usecase "Lehrveranstaltung teilen" as UCInvite
    usecase "Mensa-Angebote anzeigen" as UCMeals
    usecase "Aenderungsantrag stellen" as UCChange
    usecase "Aenderungsantrag pruefen" as UCReview
    usecase "Benutzer verwalten" as UCUsers
    usecase "Offline-Daten synchronisieren" as UCOffline
}

Student --> UCAuth
Student --> UCRole
Student --> UCContacts
Student --> UCTodos
Student --> UCSchedule
Student --> UCImport
Student --> UCInvite
Student --> UCMeals
Student --> UCOffline
Manager --> UCChange
Admin --> UCReview
Admin --> UCUsers
Admin --> UCChange
SWFR --> UCMeals
StarPlan --> UCImport

UCImport ..> UCSchedule : <<include>>
UCChange ..> UCReview : <<extend>>
UCOffline ..> UCContacts : <<include>>
UCOffline ..> UCTodos : <<include>>
@enduml
```

A.2 Logische Komponenten

```
@startuml
skinparam componentStyle rectangle
skinparam shadowing false

package "Mobile App (Expo / React Native)" {
    [Expo Router Tabs] as Tabs
    [AuthContext] as Auth
    [OfflineProvider] as Offline
    [SyncContext] as Sync
    [Client Crypto] as Crypto
    [Theming] as Theme
}

package "Backend (Express)" {
    [REST API] as Api
    [Auth Middleware] as AuthMw
    [Admin Review Service] as Review
    [Schedule Import Service] as ScheduleImport
    [Mensa Import Service] as MealImport
}

package "Persistence" {
    database "SQLite" as DB
    [Prisma Client] as Prisma
}

cloud "HFU StarPlan" as StarPlan
cloud "SWFR Speiseplan" as SWFR

Tabs --> Auth
Tabs --> Sync
Tabs --> Crypto
Tabs --> Theme
Offline --> Tabs : cached GET
Sync --> Api : replay POST
Auth --> Api : Bearer Token
Tabs --> Api : REST/JSON
Api --> AuthMw
Api --> Review
Api --> ScheduleImport
Api --> MealImport
Api --> Prisma
Prisma --> DB
ScheduleImport --> StarPlan : HTML/JSON Import
MealImport --> SWFR : HTML Import
@enduml
```

A.3 Offline-Prozess

```
@startuml
skinparam shadowing false

|Student:in|
start
:Kontakt oder To-do anlegen;
```

```

|Mobile App|
:POST an Backend senden;
if (Backend erreichbar?) then (ja)
  |Backend|
  :Daten validieren;
  :Datensatz speichern;
  |Mobile App|
  :Ansicht als synchronisiert markieren;
else (nein)
  |OfflineProvider / SyncContext|
  :Aktion lokal in Queue speichern;
  :GET-Cache fuer Anzeige nutzen;
  |Student:in|
  :Pending-Status sehen;
endif

|SyncContext|
repeat
  :Verbindung pruefen;
repeat while (offline?) is (ja)
:Queued Request mit X-StudentGo-Sync-Replay senden;

|Backend|
:Datensatz speichern;

|Mobile App|
:Queue-Eintrag entfernen;
:Listen neu laden;
stop
@enduml

```